Binary trees follow up answer key

TEACHER COPY • Covers the worksheet and student booklet

Teaching sequence and checkpoints

Allow 70–80 minutes, or split after Task 3. Tasks 1–2: 15–20 min. Task 3: 20 min. Task 4: 10 min. Task 5A: 5 min. Task 6 and exit: 15–20 min. Extensions can become a later lesson. All tasks are original formative practice; no official marks are claimed.

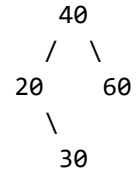
Scope: retain VB.NET and columns left=0, data=1, right=2. Use integer indexes throughout. No classes, constructors, getters or object references. Supply the table so insertion does not compete with learning recursion. Records are optional and use fields only.

Important change from the original lesson: this traversal checks p=-1 on entry and calls both child subtrees unconditionally. The old lesson checked each child pointer before making a call. Explain that both designs can work; use only the entry-check version here because it also safely accepts an empty starting tree.

Checkpoint	Ready to move on when…	If not yet secure…
After Task 2	Pupil distinguishes index 3 from data 30	Point to pointer cell, row index, then data
After Task 3	Pupil names the waiting call and resume rule	Repeat a leaf and a three-node tree with paper frames
After Task 4	Pupil explains the guard and the two child arguments	Keep using the recipe before completing code

Task 1

A. After 40: root only. After 20: left child of 40. After 60: right child of 40. After 30: right child of 20. Accept correct incremental sketches; preserve left/right placement.



B. Root 40; leaves 30 and 60; subtree at 20 contains 20 and 30. C. 30 < 40, but 30 > 20. Binary trees have at most two children; only an ordered binary tree adds the ordering rule.

Task 2

Index	Left (0)	Data (1)	Right (2)
0	1	40	2
1	-1	20	3
2	-1	60	-1
3	-1	30	-1

A. RootPointer=0. B. Indexes 0,1,3; data 40,20,30. C. TreeArray(1,2)=3 is a pointer; TreeArray(3,1)=30 is the child's data. D. Root is 2:7, left child 1:4, right child 0:9. Both children are leaves.

Task 3

Drawing: root 1:25; left leaf 2:10; right leaf 0:15. It is not a BST because right child 15 is less than 25. In-order still means left, node, right.

Event	Active p	What happens / where next?	Output added
1	1	Call InOrder(2); wait at rule 3	—
2	2	Call InOrder(-1); wait at rule 3	—
3	-1	Return to p=2, rule 3	—
4	2	Output data	10
5	2	Call InOrder(-1); wait at rule 5	—
6	-1	Return to p=2, rule 5	—
7	2	Return to p=1, rule 3	—
8	1	Output data	25
9	1	Call InOrder(0); wait at rule 5	—
10	0	Call InOrder(-1); wait at rule 3	—
11	-1	Return to p=0, rule 3	—
12	0	Output data	15
13	0	Call InOrder(-1); wait at rule 5	—
14	-1	Return to p=0, rule 5	—
15	0	Return to p=1, rule 5	—
16	1	Return to original caller	—

Accept equivalent event descriptions. Each -1 invocation must return before its caller resumes. C. Active p=2; waiting call InOrder(1), resuming at rule 3. D. Output 10,25,15. Each return ends one invocation; earlier calls still have instructions left.

Task 4

A. (1) -1: missing subtree; (2) Exit Sub: return before array access; (3) 0: left child index; (4) 1: node data; (5) 2: right child index. B. InOrder(RootPointer); -1 outputs nothing. C. Output 20,30,40,60.

```
Sub InOrder(ByVal p As Integer)
    If p = -1 Then
        Exit Sub
    End If
    InOrder(TreeArray(p, 0))
    Console.WriteLine(TreeArray(p, 1))
    InOrder(TreeArray(p, 2))
End Sub
```

Just before outputting 30: active InOrder(3). Under it, InOrder(1) is waiting after its right call and will finish. Under that, InOrder(0) is waiting after its left call and will output 40. The empty left call from 3 has already returned. D. Column 1 supplies data, not an index. Data 20, for example, would attempt to access outside this four-row array.

Task 5

A. First mistake: line A reads the array before checking -1, so an empty call attempts row -1 and crashes. Second: after moving the guard first, output before the left call gives pre-order, not in-order. Repair by placing the guard first and output between the recursive calls. Use the complete InOrder above (renaming recursive calls consistently if retaining Broken).

```
Sub PostOrder(ByVal p As Integer)
    If p = -1 Then
        Exit Sub
    End If
    PostOrder(TreeArray(p, 0))
    PostOrder(TreeArray(p, 2))
    Console.WriteLine(TreeArray(p, 1))
End Sub
```

B/C. Post-order output: 30,20,60,40. A node is output after both of its subtrees have finished. The root therefore appears last.

Task 6 and exit check

Test	Expected output, one value per line
InOrder(RootPointer)	20, 30, 40, 60
InOrder(1)	20, 30
InOrder(2)	60
InOrder(-1)	No output
After replacing 30 with 35	20, 35, 40, 60
PostOrder(RootPointer)	30, 20, 60, 40
PostOrder(2)	60
PostOrder(-1)	No output

B. p=3, with calls 1 and 0 waiting (see Task 4). After 3 finishes, call 1 finishes, then call 0 outputs 40. C. Only data changes: 35 fits the same position (greater than 20, less than 40), so the existing links still describe the BST. Exit: InOrder(2), output 5,8; resume at the root's output line with p=2.

Optional record translation

```
For i As Integer = 0 To 3
    Nodes(i).LeftPointer = -1
    Nodes(i).RightPointer = -1
Next
For i As Integer = 0 To 3
    Nodes(i).LeftPointer = TreeArray(i, 0)
    Nodes(i).Data = TreeArray(i, 1)
    Nodes(i).RightPointer = TreeArray(i, 2)
Next
```

```
Sub InOrderRecords(ByVal p As Integer)
    If p = -1 Then
        Exit Sub
    End If
    InOrderRecords(Nodes(p).LeftPointer)
    Console.WriteLine(Nodes(p).Data)
    InOrderRecords(Nodes(p).RightPointer)
End Sub
```

Same outputs as Task 6. Integer fields default to 0 in VB.NET; explicitly initialise child pointers to -1. All four records here are populated; unused rows in a larger array must not be linked into the tree.

Complete runnable model

The companion Model.vb includes post-order and records. This minimal main program plus the InOrder procedure below is sufficient for the core lesson.

```
Module BinaryTreePractice
    Dim TreeArray(,) As Integer = {
        {1, 40, 2},
        {-1, 20, 3},
        {-1, 60, -1},
        {-1, 30, -1}
    }
    Dim RootPointer As Integer = 0

    Sub Main()
        InOrder(RootPointer)
        ' Other tests: InOrder(1), InOrder(2), InOrder(-1)
    End Sub

    Sub InOrder(ByVal p As Integer)
        If p = -1 Then
            Exit Sub
        End If
        InOrder(TreeArray(p, 0))
        Console.WriteLine(TreeArray(p, 1))
        InOrder(TreeArray(p, 2))
    End Sub
End Module
```